ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/356452360

Edge MLOps: An Automation Framework for AIoT Applications

Conference Paper · October 2021

DOI: 10.1109/IC2E52221.2021.00034

CITATIONS
68

READS
1,757

4 authors, including:

Kimmo Ahola
VTT Technical Research Centre of Finland

33 PUBLICATIONS **231** CITATIONS

SEE PROFILE

All content following this page was uploaded by Magnus Westerlund on 30 November 2021.

The user has requested enhancement of the downloaded file.

Edge MLOps: An Automation Framework for AIoT Applications

Emmanuel Raj
TietoEVRY
Helsinki, Finland
emmanuel.raj@tietoevry.com

David Buffoni
TietoEVRY
Stockholm, Sweden
david.buffoni@tietoevry.com

Magnus Westerlund
Arcada University of Applied Sciences
Helsinki, Finland
magnus.westerlund@arcada.fi

Kimmo Ahola
VTT
Espoo, Finland
kimmo.ahola@vtt.fi

***Abstract*—Artificial Intelligence of Things (AIoT) is the combination of artificial intelligence (AI) technologies with the Internet of Things (IoT) infrastructure to achieve more efficient IoT operations and decision making. Edge computing is emerging to enable AIoT applications. In this paper, we develop an Edge MLOps framework for automating Machine Learning at the edge, enabling continuous model training, deployment, delivery and monitoring. To achieve this, we synergize cloud and edge environments. We experimentally validate our framework on a forecasting air quality situation. During validation, the framework showed stability and automatically retrained, integrated, and deployed models for specific environments when their performance deteriorated under a certain threshold.**

***Keywords*-Edge Computing, Machine Learning, IoT, 5G Networks, AI, Automation, Digital Transformation, MLOps.**

I. Introduction

In recent years, Internet of Things (IoT) has grown to become ubiquitous and this has led to a need for the development of new process support methods to operate big data environments at the edge and a demand for low latency communication, enabled by technologies such as 5G Networks. This shift in infrastructure helps to digitize physical spaces and enables real-time decision making using Artificial Intelligence [1]. Artificial Intelligence of Things (AIoT) is the combination of artificial intelligence (AI) technologies with the Internet of Things (IoT) infrastructure to achieve more efficient IoT applications and improved local decision making [2]. Edge computing enables generating insights and making decisions at the data source, reducing the amount of data sent to the cloud and central repository [3].

Performing Machine Learning (ML) inference at the edge has several benefits compared to traditional cloud-based machine learning [3]. In addition to reducing the amount of data exchanged, Machine Learning models tailored to specific IoT devices can be used to perform tasks customised to their respective environments [4], which is not the case with the cloud-based solutions. In some use cases, consented collection and processing of sensitive personal data is required or useful for the purpose of improving an individual's daily routine [5]. For this purpose, edge computing can facilitate data and privacy-preserving processing in accordance with privacy laws. Additionally, as we note in Section VIII, the dimensions of privacy can further be enhanced by methods such as Federated Learning, in which no edge node or centralised computer exchanges data, but rather model parameters are shared (weights and biases) [6]. Traditional ML at the edge, has also some limitations. For instance, machine learning models are often trained centrally and deployed manually and individually to the edge nodes. This process of developing, deploying and monitoring Machine Learning models needs to be automated to be scalable. Security is an essential challenge in IoT and edge computing, as edge nodes are prone to attacks [7], [8]. In most cases, privacy and compliance are highly essential and the edge-cloud setup has to comply with country specific data privacy laws. In order to fully leverage the capabilities of AIoT, to facilitate automated decision making at the data source, a framework for seamlessly integrating the two paradigms is necessary. The objective of the paper is to investigate and develop methods for enabling automated machine learning at the edge that can be used by enterprises or industry.

Hence the focus of the research is to answer the question: *How can a framework that integrates continuous delivery and continuous deployment of machine learning models at the edge be implemented using state-of-the-art tools and methods?* Drawing on our experience of building large-scale and real-world machine learning pipelines, utilizing Machine Learning Operations (MLOps) [9], [10], we present a new automated framework for Machine Learning Operations at the Edge (Edge MLOps). The main contributions of the paper are the proposed Edge MLOps framework, a use-case verification for a fully automated and scalable process for training, deploying and monitoring IoT data and edge node operations in real-time. Finally, we discuss an extended architecture for privacy considerations utilizing federated learning.

The paper is structured so that we first review the literature in Section II, then we describe the Edge MLOps framework in Section III. Section IV presents the setup for experimenting our framework. Section V and Section VI disclose the empirical choices made for carrying out the experiments. Finally, in Sections VII and VIII we discuss a future extension for improving privacy and summarize the results.

II. Related work

A large number of AI applications such as Computer Vision, Autonomous Driving, and AIoT can be realised in edge computing environments. Placing AI services close to

the data source can improve user experience by reducing the latency, the dependence on network connectivity and the communication cost [11]. Sato [9] introduced MLOps as a part of DevOps best practices for ML. In [12], MLOps is presented as a solution for handling and improving AutoML and feature-specific ML engineering, such as explainability and interpretability. Further studies have described how to perform inference of AI and more specifically Machine Learning (ML) on the edge [13], [14], [15]. For improving the efficiency of ML models' inference on edge devices, efforts have been made to optimize Mobile Edge Computing (MEC) workloads by optimizing operations on the cloud and edge nodes [16], [17]. For optimizing the training of ML models on edge devices, there are attempts to build a framework to train machine learning models on resource-constrained IoT edge devices [18]. However, the training and deployment of the ML models is often done manually and some efforts have been made to scale and automate this process.

The discipline of integrating DevOps principles and ML is at its beginnings and the proposed solutions are still often specific to particular industries or use-cases [19], [20], [21]. There are also other complementary Ops-based methods being developed such as DataOps, that consider streamlining the data flow between services, for instance within a microservice architecture [22], [23]. There exist some attempts of building a framework for the lifecycle management of AI applications [24]. Most of them, are based on cloud solutions without any focus to edge environments (e.g. [25]). Our solution proposes a framework for automating the AI workflows by combining cloud and edge environments. A similar work to ours is the framework proposed by [26], [27] where cloud and edge platforms are combined together. The main difference with [26] is the authors' focus on training ML models on the cloud and deploying them on edge devices while our work focuses on automating the deployment process. In [27], the authors proposed to trigger the process by model versioning while in our work, models' performance drops or data drift are considered. Moreover, our framework can be easily extended to a Federated Learning setting where data privacy and security is mandatory.

III. Edge MLOps Framework

To enable a use case of individually trained AIoT applications on edge nodes, as further discussed in Section IV, we present an Edge MLOps framework to automate and operationalize the Continuous Delivery (CD) and Continuous Integration (CI) of ML models to the nodes. Usually, deploying ML models happens in two distinct steps. The training step fits the parameters of an algorithm to a given dataset. Once the training is complete, the trained model can be used for inferring new data points during inference [28].

Distributed edge devices may have several advantages over a conceptually centralized cloud solutions. For instance, it allows for processing data in real-time, specialized modelling per node, minimized communication needs, and improving data privacy protection. However, edge devices tend to have low processing power, small storage capacity, and varied specifications. Training machine learning models on edge devices is a difficult challenge as ML can be intensive in terms of computation, memory, and storage. However, edge devices connected to a mains power source are often sufficient for ML inference.

To provision computation resources for machine learning model training, an alternative is to use a cloud platform. Cloud platforms have several advantages in that they provide powerful processing and computation resources, massive storage and efficient management. However, they may be inadequate because of insufficient data privacy, high bandwidth costs and reliance on network connection. For the interested reader, a more complete description of the strengths and weaknesses of both platforms can be found in [26]. To combine the best of both worlds, we present an ML lifecycle management framework, Edge MLOps, that uses a hybrid edge and cloud solution.

The overall requirements are addressed by the framework in the following manner using the cloud platform and edge devices:

Cloud platform

- ML pipeline for model training and retraining
- CI and CD
- Source code management
- Data Storage
- Fleet Analytics (dashboard)

Edge devices

- Continuous Integration and data streaming with IoT devices
- Continuous Integration and Delivery from cloud to edge
- Hyper-personalization
- ML inference

The framework facilitates the above requirements and functionalities in a modular way. On a high level, there are two main layers of the framework. 1) The Cloud Orchestration layer which runs on the cloud platform is dedicated for ML model training, retraining, CI-CD jobs, centralized data storage, source code management, analytics dashboard, and data and model governance. 2) The Edge Inference layer is dedicated for orchestrating ML inference and operations for IoT devices, such as continuous integration and streaming data from the IoT devices. To enable robust and low-latency network communication between these two layers, 5G Networks or WLAN can be used for communicating continuous delivery triggers, and to send and receive data as needed. An overview of our proposal for Edge MLOps is shown in Figure 1.

The advantages of such a framework are a high level of decoupling through modularity between the two layers, and utilization of the strengths of both platforms. In the following sub-sections we describe the two layers, Cloud Orchestration and Edge Inference.

A. Cloud Orchestration

In the Cloud Orchestration layer, multiple services are running on the cloud to perform parallel jobs that adds

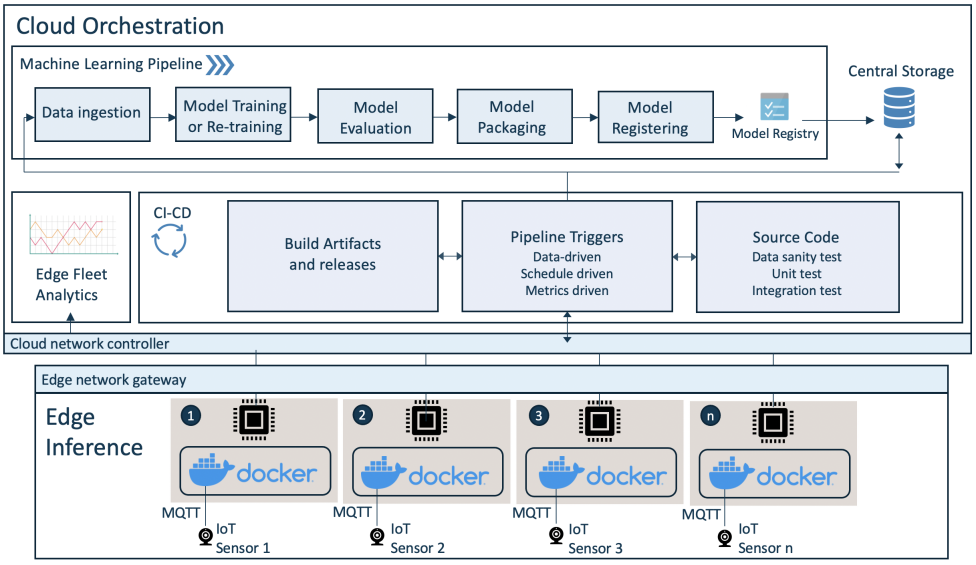

Fig. 1. Proposed Framework Architecture for Edge MLOps for AIoT Applications.

additional complexity to the whole process. ML modeling is used to enable fast and reasonably accurate predictions for newly generated data, which is often the case in an AIoT setting. We train our models on the cloud due to the availability of high performance compute resources and we use an ML pipeline to facilitate the ML models' training on the cloud. Consequently, machine learning steps for handling specialized ML models per device include training, testing, selecting best model candidates for deployment, monitoring performance in production, and maintaining prediction quality. In a scalable setting these steps become tedious tasks that cannot be manually addressed. Instead, we propose to handle them with a Cloud Orchestration layer that is composed of four functionalities:

1) **Machine Learning Pipelines**: An ML pipeline which builds and deploys models is usually proposed by cloud vendors as a basis for cloud-based MLOps. Such a service provides computational resources and data storage on-demand to enable training of machine learning models. Typically, this service also comes with an ML model repository and container storage to enable faster deployment. We use the ML pipeline to train and retrain ML models. To train and validate an ML pipeline for a specific dataset, we consider these five steps:
   - Data Ingestion: The ingestion script procures data (based on parameters) and versions the data which will be used for ML model training. As a result of this step, any experiment (*i.e.* model training or re-training) can be audited and traced back.
   - Model Training and retraining: a script that performs all the traditional steps in ML such as data pre-processing, feature engineering, feature scaling before training or retraining any model. Usually, ML models have a set of hyperparameters to tune for fitting the model to the dataset (training set). This step can be done manually, but efficient and automatic solutions such as GridSearch or RandomSearch [29], exist.
   - Model Evaluation: the inference of the trained model is evaluated according to different metrics on a separate set of data points named test set. The output of this step is a report on the model performance.
   - Model Packaging: once the trained model has been tested, the model is serialised and packaged into a container for standardized deployments.
   - Model Registering: the model candidate, which has been serialised, is registered and stored in the model registry from where it is ready for quick deployment into an edge device.
2) **Storage**: For storing structured and unstructured data, services are commonly available in a cloud vendors catalogue. Cloud storage solutions are usually autoscaled depending on the demand which makes them convenient and efficient to use. To be able to keep track of the model versions and trace back if needed, data versioning and governance is necessary.
3) **Fleet Analytics**: Fleet analytics is a data analysis and monitoring tool for data generated from the edge and IoT devices and also the predictions made by the deployed models. Cloud vendors usually propose some ready-made tools for checking the health and performance of IoT and edge devices. A custom dashboard can also be created to monitor these insights in real-time [30].
4) **CI/CD**: Continuous Integration and Continuous Deployment pipeline enables continuous delivery to the edge layer. The goal of the CI/CD pipeline is to deploy, retrain and maintain models while maintaining end to

end traceability (e.g. maintaining logs, tracking versions of models deployed as well as datasets and source code used for model training). The CI/CD pipeline also enables triggers to perform necessary jobs in parallel, build artefacts and release for deployment to edge devices.

B. Edge Inference

The Edge Inference layer focuses on orchestrating operations for the IoT and the edge devices. It enables real-time machine learning inference and works in sync with the Cloud Orchestration layer for deploying the ML models to the edge and transferring data from devices for training and testing purposes. More details on each capability of the Edge Inference layer are given below:

1) Continuous Integration and Delivery from cloud to edge: Continuous delivery to the edge is driven by a high speed and low latency network for stable and robust communication and operations. We propose using a private network (*e.g.* WLAN) to support the communication between edge devices, IoT devices, and Cloud layer. Inside this network, edge devices communicate and infer data from the sensors using the MQTT protocol. For stability and standardization reasons, all these operations are run inside containers, (*e.g.* Docker containers). Continuous deployment pipeline facilitates model deployments in containers to the edge, data transfer, and monitoring (ML models and edge devices).

2) Hyper-personalization at the edge: Edge and IoT devices in a network can be used in different environments, consist of different hardware, and be located in separate geographical locations. In many cases, the devices need to perform tasks customized to their respective environments. In such cases, edge nodes can enable customization for each device using a custom ML model. This way, ML models deployed at the edge can optimize and retrain themselves when needed, while constantly learning to serve better. We train ML models on the cloud (see sub-section III-A), using data from respective rooms or environments to train customized ML models.

However, devices often come with different hardware architectures (e.g. arm64X, x86 etc) or sensor setups, so to personalize a model for deployment demands customization for specialized data and the variety in the edge hardware. By using appropriate docker containers tailored to each architecture we improve hardware stability and standardization among nodes in the network.

3) Automated Machine Learning at the edge: Machine Learning inference and monitoring are automated as part of Continuous Deployment operations. A periodic trigger from the CI-CD process in the Cloud Orchestration layer is implemented to invoke the monitoring feature in the edge devices, to evaluate model drift and to replace or retrain the existing ML model with an alternative [31]. With this approach, the whole process of ML inference at the edge is automated in real-time.

IV. Experimental Setup

To validate our Edge MLOps Framework discussed in Section III, we carried out experiments on a VTT Technical Research Center campus in Finland where IoT devices and edge devices were setup as illustrated in Figure 2. The aim of the experiment is to continuously monitor air quality and conditions in 3 rooms, each of them equipped with differing IoT devices.

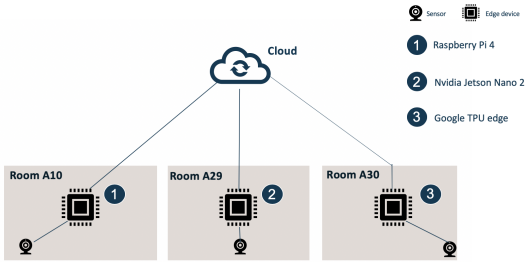

Fig. 2. Overview of the experimental setup for 3 rooms. Each room is equipped of an IoT and an edge devices.

Below we present the different Hardware, Software and Network solutions we used in our experiments. The choice we made was to use industry standard solutions to make our framework easily replicable by the industry.

1) Hardware Tools: For edge devices we used Raspberry Pi 4, Nvidia Jetson Nano 2 and Google TPU edge.

2) Software Tools: For software development, we have chosen common data scientist's tools for the tech stack to do data analysis, model training and deployment, and monitoring - Python, Linux and Docker. Docker containers are used to deploy applications in runtime and are a standard in industry. Microsoft Azure is used as the cloud service because it incorporates a plethora of reliable services for Ops and Edge computing.

3) IoT devices and Networking: In this work, we use Treon Node IoT sensors to establish an office air quality monitoring system. In addition to temperature and humidity, the sensor also measures volatile organic compounds (VOCs), producing estimates of air quality. The sensors form a Wirepas sensor mesh network, which features adaptive routing and diagnostics monitoring for both the nodes and the network. A node sends measurements every five minutes, and the sensor mesh network relays the data to a gateway. The gateway then uses the Message Queue Telemetry Transport (MQTT) protocol, a robust real-time messaging protocol for IoT applications [32], to send the information to a Mosquitto MQTT broker running in a VTT Technical Research Center test network. The MQTT protocol was chosen because it is widely used in IoT environments and it was the basis of the edge environment already installed as part of a different research project. The edge nodes run behind a NAT router, also in the test network, and they have access to the IoT sensor data via the MQTT broker.

4) Central Storage: For storing all of our data such as sensor outputs, ML models, training dataset, artifacts and

logs, we provisioned an auto-scalable storage on the cloud (Azure blob storage) to serve our storage needs. To handle data privacy, we provisioned our central storage in the region of our closest proximity with a restricted access to the authorized services and parties only. To achieve that in practice, we used VLAN and subnet configuration. Additionally, the storage is provisioned with full end-to-end data lineage allowing data traceability.

In the following Sections V and VI we detail our work as two separate cycles, design and empirical, in accordance with the design science method [33].

V. Design Cycle

To design a robust and scalable framework for Edge MLOps for AIoT applications we did multiple iterations and assessments as part of the design cycle. Firstly, we mapped out the data flow as shown in Figure 3 for the use case (based on the requirements discussed in Section III). The data flow is observed in four phases - Data Processing and Model Training, Model Selection, Inference and Fleet Analytics and Actions. In the Data Processing and Model Training phase, ML models are trained using the training data (collected from IoT Sensors for a period of 3 months). The trained models are packaged and registered into the Model Registry. For Model Selection, we select the best models using the quantitative model recommendations (as described in section VI). The selected models are then deployed using containers in the respective edge devices. The ML model detects any anomalies (bad air quality predictions) via the container port. The predicted anomalies are reported to the Edge Fleet Analytics dashboard which raises an alarm notifying the guardian.

We assessed services from Microsoft Azure with the goal to facilitate various steps in the framework such as continuous delivery, deployment and monitoring on edge devices, cloud to edge communication orchestration, etc., to name a few. Then, suitable services were chosen based on the maturity of the service for the experiment and efficiency for enabling orchestration and automated pipelines for synergy between edge and cloud. The selected services were Azure ML Service (for MLOps), Azure DevOps (for Source Code Management), Azure IoT Central (for Fleet Analytics), Power BI (for Fleet Analytics) and Azure blob storage (for data and models storage).

1) Continuous Integration from IoT to Edge: We communicate with the IoT devices from edge devices and establish continuous integration between the respective IoT devices and the edge devices. Communication between IoT and Edge Devices is established using the MQTT protocol which has become a standard pub/sub platform for IoT applications.

2) Continuous Integration from IoT to Cloud: Two processes were implemented for Continuous Integration, delivery and deployment as shown in Figure 4. These processes were setup to facilitate the end to end Continous Integration from IoT devices to the Cloud through the edge devices. The two scripts are running inside a Docker container and orchestrate the incoming data and the ML pipelines. For system resilience we configured an automated restore to default firefighting measure. Whenever a node failure is detected the edge device is redeployed with the respective docker container to restore the default state.

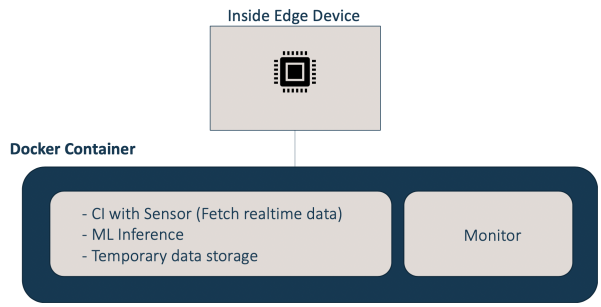

Fig. 4. Docker container deployed in each edge device

- **Ingestion, Inference and Serialization Process**:
  - This process enables and maintains continuous integration of sensor to edge by fetching data in real

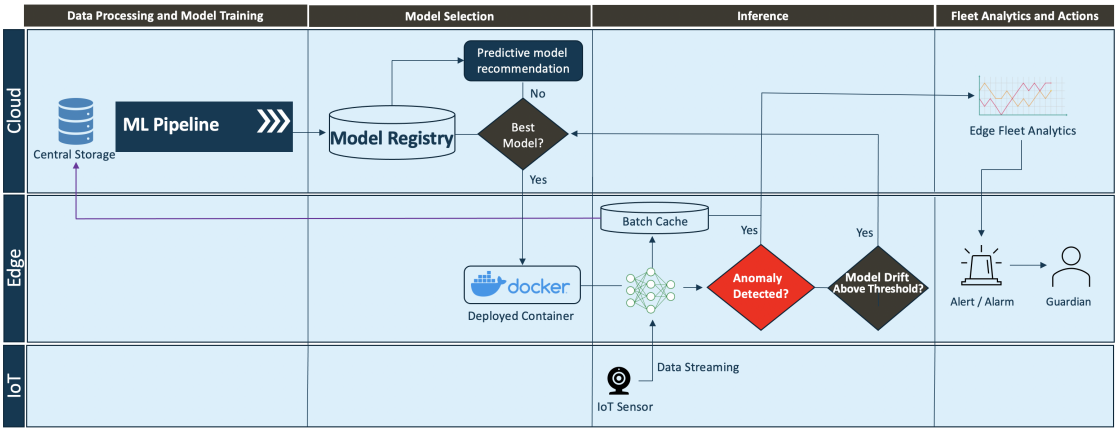

Fig. 3. The dataflow of the use case.

time. Received data is pre-processed, cleaned and formatted for ML inference.
  - A trained ML model (from the Cloud) runs inside the docker container consumes the new data and forecasts the air quality in the next 15 minutes.
  - Processed data from the IoT device and forecasts are concatenated together and appended to a Comma Separated Value (CSV) file temporarily stored in the docker container.

- **Time Trigger Process**: A scheduled time trigger is implemented for evaluating potential ML model drift in performance comparing actual and future predictions. If the evaluation performance difference is greater or equal to a predefined threshold, then, the process calls to look for and deploy an alternative model from the Cloud.

These two processes make the bridge between IoT sensors and the Cloud and enable a Continuous Integration between the IoT devices and the Cloud. Thanks to that, Continuous Delivery and Deployment is possible.

3) Continuous Delivery and Deployment for Edge: In this section we will look into important aspects of Continuous Delivery and Deployment on the edge. We rely on the Azure DevOps service for the CI-CD pipeline. After configuring a secure connection through SSH with Azure DevOps and each edge device, the CI-CD pipeline orchestrates the Cloud services for training the models (Azure ML Service) and storing them (Azure Blob Storage).

In the following, we describe each phase of CI-CD pipeline:

1) **Release**: A release is triggered to monitor edge devices to check for model performance in real-time. A new model is trained using both real-time data and previous data snapshots. In our experiments, time-based and manual triggers were implemented but other options are possible such as at the source code commit or new trained models.
2) **Monitor edge and deploy**: the aim is to monitor the ML models deployed on edge devices by evaluating their drift. Processing of the data stored on each edge is done parallelly. In practice, we evaluate the air quality forecast (for the next 15 minutes) and the actual air quality records with the Root Mean Square Error (RMSE) [28]. In case the RMSE is greater or equal to a fixed and empirically defined threshold, a call is made to replace the existing model by a new version from the Model Repository. The approach is generic and can be used with other evaluation metrics and other threshold values. This ensures the Continuous Deployment of ML models to the edge.
3) **Model Retrain**: When a call for replacing a model is made, the existing model is fine-tuned on the Cloud with latest real-time data to keep it up-to-date. Then, the model is stored in the Model Repository for future deployments.

VI. Empirical Cycle

We now describe how we empirically evaluate and validate each step of the Edge MLOps framework.

A. Machine Learning Operations with Cloud Orchestration

We first experiment with all the steps of the Machine Learning pipeline on the Cloud Orchestration layer, where Machine Learning Operations are done (see Figure 5).

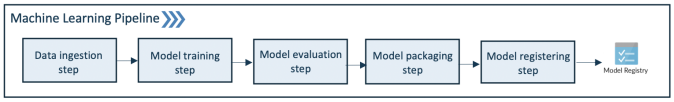

Fig. 5. Machine Learning pipeline on the Cloud Orchestration layer.

We train our models on the cloud due to the availability of high performance compute resources and we use an ML pipeline to facilitate the ML models training on the cloud. For this purpose, we use Azure Machine Learning service as a platform where we can provision compute, storage and the needed infrastructure on request for ML Operations. Using Azure ML service, we can train, manage, deploy, and audit models (incl. model traceability with data and source code versioning support from training). Models are trained separately (using respective datasets of rooms) to be deployed in rooms in the respective edge devices. We train a dedicated model for each room using the data from the particular room where the edge device is set up (in which we deploy the custom trained model). A dedicated model for each room or device will enable model hyper-personalization (as shown in section I) in the respective environment [24].

1) Data Ingestion Step: Data ingestion and Data Exploration are done in this step. For the experiment, data has been collected for 3 months, starting from 15th October 2019 to 15th January 2020 from 26 different IoT devices generating a total of 537873 event. Each event contains 13 parameters which are presented in Table I

Event variables list			
timestamp (datetime)	name (string)	room (string)	room type (string)
floor (string)	air quality altered (float)	air quality unaltered (float)	pressure (float)
ambient light (float)	humidity (float)	IAQ accuracy altered (float)	IAQ acc. unaltered (float)
temperature (float)			

TABLE I: Event description. IAQ stands for Indoor Air Quality.

We collaborated with the Finnish Technical Research Center who are interested in forecasting the air quality in real-time. The aim is to train ML models to be able to predict the air quality in the rooms.

We performed data analysis on the collected data and we observed that 3 rooms out of 26 had a high variance in their air quality (2 of them were meeting rooms). This would force our framework to train new forecasting models frequently to

maintain good prediction performance. We report in Table II the descriptive statistics of the selected 3 rooms.

Selected Rooms			
Room name	Room type	Unhealthy air quality frequency	Avg. air quality
Room A10	Office room	2033	61.92
Room A29	Meeting Room	2205	61.40
Room A30	Meeting Room	1085	55.45

TABLE II: Descriptive statistics for air quality in selected rooms. Unhealthy air quality frequency is the number of times the value provided by the IoT devices is greater than a threshold (e.g. 150+).

Our goal is to provide a satisfactory forecast of the air quality of each room. To achieve this, we need to perform a Time Series Analysis [34] to understand better the nature of the different time series and how to model them. We plot the room time series in Figure 6 which show a non-stationary pattern since mean, variance and covariance are observed to be changing over time. It may be due to several factors such as trends, cycles, random walks or combinations of the three.

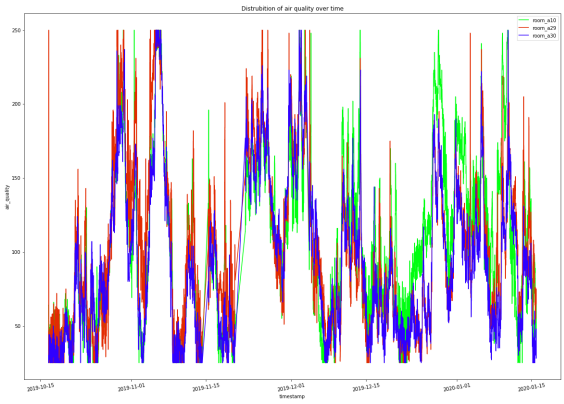

Fig. 6. Air quality Time Series for the 3 selected rooms.

2) Machine Learning Step: Based on the collected Time Series, the goal is to train ML models for each room to forecast the air quality for the next 15 minutes. To achieve this, we have to perform Feature Engineering [28] to prepare data for the ML models. We follow the Supervised Machine Learning paradigm for predicting the air quality. Supervised Machine Learning models learn an objective function which for any input data produces a prediction. The goal of ML algorithms is to find a solution minimising the number of prediction errors. Prediction errors are evaluated with a loss function comparing the predictions and the targets. In our case, the targets are the Time Series which have been shifted 3 steps ahead (15 minutes). On the parameters generated by IoT devices we analysed the ones which have a correlation with the air quality 15 minutes ahead. We found that the air quality static, ambient light, humidity, iaq accuracy static, pressure and temperature were correlated to the targets. After using a Z-score scaler, these parameters are provided to the ML algorithms. We experimented with these 4 supervised ML models for producing forecasts: Multiple Linear Regression (MLR), Support Vector Regressor (SVR), Extreme Learning Machines (ELM) and Random forest Regressor (RFR) (see [28] [35] [36] [37]). The model hyperparameters are selected through gridsearch on a Time Series 10 Fold Cross-Validation splits.

3) Model Evaluation Step: The best model candidates are selected based on the lowest RMSE averaged over the folds. Detailed results are reported in Table III.

Other feature extractions or ML models could have been used to provide more accurate Time Series forecasts but we are interested in the ML model lifecycle management in its whole.

Model Training Results			
Room name	Algorithm	Cross Validation RMSE (train)	Test RMSE
Room A10	MLR	5.020	5.875
Room A10	ELM	6.325	6.208
Room A10	RFR	10.710	9.987
Room A10	SVR	6.046	5.977
Room A29	MLR	5.362	4.158
Room A29	ELM	11.202	4.223
Room A29	RFR	11.676	9.208
Room A29	SVR	8.073	4.176
Room A30	MLR	3.648	3.551
Room A30	ELM	7.920	3.895
Room A30	RFR	9.686	7.720
Room A30	SVR	5.177	3.55

TABLE III: Model training results.

4) Model Packaging Step: Once the models have been trained, they are serialised and packaged to be ready for inferring new incoming data on the respective edge devices. For feature engineering we store the scaling variables into a serialized file to be reused when models are deployed on the edge. The trained models are serialised in the Open Neural Network Exchange (ONNX) format. ONNX is an open ecosystem for interoperable AI models, it enables model interoperability and serialisation of ML and deep learning models in a standard format. We package the scaling variable, serialized ONNX model and monitoring script (which monitors the model drift) into a container and register/store the container in the model registry (where we store and version our model container) which is part of the Azure ML service. By packaging scaling variables and models into containers we can pull, deploy and run these containers on edge devices such as Raspberry Pi 4, Jetson Nano and Google TPU edge.

5) Model Registering Step: Packaged models are registered and stored in the ML model store for quick deployments.

B. Edge Inference

On each edge device, for each new event computed by the IoT devices, the ML model predicts the future air quality for the current room. As a reminder each model is replaced by an updated version when a performance drop is noticed. We measured for 45 days the activity of the Edge MLOps framework by counting the number of deployed models and

the number of failures. A total of 23 models have been changed or continuously deployed on respective edge devices without reporting any error. Table IV lists the deployed models by the Edge MLOps framework. We notice that for all new deployed models, the RMSE decreases meaning that the updated model performs better than the previous one. In addition, our implementation successfully performed for the different edge devices without showing any compatibility issues with our framework.

Realtime machine learning inference at the edge					
S.no	Date of model change	Edge Device	Deployed Model	Model Drift (RMSE)	Model Retrain (RMSE)
1	15-03-2020	Jetson nano 2	ELM	16.39	4.1
2	16-03-2020	Google TPU edge	RFR	14.23	6.3
3	16-03-2020	Raspberry pi 4	MLR	11.91	4.3
4	17-03-2020	Raspberry pi 4	ELM	13.27	8.1
5	22-03-2020	Jetson nano 2	SVR	22.32	6.2
6	24-03-2020	Google TPU edge	RFR	17.11	4.4
7	27-03-2020	Raspberry pi 4	MLR	16.22	4.7
8	29-03-2020	Jetson nano 2	ELM	30.28	8.2
9	30-03-2020	Google TPU edge	SVR	18.12	5.4
10	05-04-2020	Raspberry pi 4	MLR	12.92	3.2
11	10-04-2020	Jetson nano 2	SVR	17.21	5.2
12	11-04-2020	Google TPU edge	MLR	13.42	4.7
13	13-04-2020	Jetson nano 2	ELM	27.29	5.3
14	17-04-2020	Google TPU edge	RFR	17.46	6.9
15	19-04-2020	Raspberry pi 4	SVR	16.32	5.1
16	19-04-2020	Google TPU edge	MLR	11.91	3.4
17	21-04-2020	Jetson nano 2	ELM	23.26	7.3
18	22-04-2020	Google TPU edge	RFR	16.92	7.2
19	24-04-2020	Raspberry pi 4	SVR	17.87	5.2
20	25-04-2020	Google TPU edge	MLR	13.92	5.2
21	25-04-2020	Jetson nano 2	SVR	19.21	7.9
22	26-04-2020	Raspberry pi 4	ELM	23.57	6.4
23	26-04-2020	Google TPU edge	SVR	18.21	5.5

TABLE IV: Results of the Machine Learning inference at the edge.

VII. Results

For the purpose of this study, validation of the design cycle was done over several iterations. We assessed cost, energy and operational efficiency of edge vs cloud machine learning inference based on our experiments. For benchmarking, we used a single edge device and compared it against a compute resource on the cloud. For edge devices we used Raspberry pi 4's and for a cloud compute, we used a data science virtual machine DS2 v2 (on Azure Cloud). Table V shows the results of monitoring the models deployed in 10 edge devices (Raspberry Pi 4) compared to 1 cloud node (a data science virtual machine DS2 v2 on Azure Cloud) for a timeline 30 days or a month. Based on our experiments the framework setup provided an improvement of resource usage by almost 9 times compared to the same experiment performed on cloud computing using micro services. Our calculations are based on power consumption references from Raspberry Pi and Microsoft (for data science virtual machine DS2 v2) [38], [39]. There was an overall 9-times cost reduction and a 9-times increase in inference speed. In addition to that, 22% of the incoming data from IoT devices were pruned (to send only essential data to cloud for storage) reducing storage costs in the cloud up to an order of magnitude. Similar studies are described in [11] suggesting the energy and cost efficiency for inference at the edge.

Edge vs Cloud inference based on the experiments		
	Edge devices (10)	Cloud node (1)
Device	Raspberry pi 4	DS2 v2 (Azure)
Computation	40 vCPUs (4x10)	2 vCPUs
RAM	40 GB (4x10)	7 GB
Temporary storage	640 GB (64 GB/device)	14 GB
Data pruned	22 %	0 %
ML inference/minute	1/device	10
Avg. inference time	0.2 seconds	2.2 Seconds
Total cost/month	$ 10/month	$ 93/month

TABLE V: Quantitative analysis - Edge vs cloud based on the experiments.

We evaluate the robustness of our framework at the different stages. At a 5-minute interval, IoT devices sent data to edge devices without any failures. During the 45 days of experiment, a time trigger was executed every day at 12:00 to monitor the model drift of each deployed model. A total of 135 time triggers were executed successfully without any failure. In addition to them, 27 manual triggers have been executed in a random fashion without any failure. On the Cloud layer, 23 ML models have been successfully retrained when their performance was degraded. An aggregate of 38 MB of data was collected without any interruptions or data leakage. Data collection and storage pipeline worked and no interruptions were detected. These results indicate that the proposed solution is feasible and stable. Edge MLOps is scalable to multiple edge devices (depending on the use case) and is confined to the infrastructure (cloud and networking) and tools used to implement the framework and perform experiments. Our framework is generic enough to be applied on any AIoT application such as healthcare, manufacturing, telecommunication, energy, etc.

Our framework proposes a stable and automatic way to train models on the cloud and deploy them on edge devices. This results in a stable and robust pipeline. Automation in the ML model lifecycle management is worth considering as it saves times and manual labour. Manual deployment of models can be prone to errors due to human intervention. This risk is mitigated by our approach.

VIII. Future Directions

A. Towards a Fault-tolerant Edge MLOps Framework

In our experiments, we have been able to deploy 23 models without any errors for 45 days but it is worth noting that we may encounter exceptions in the long run. For example, model training may fail or model inference may not be as expected which may result in unexpected behavior of the system. To tackle this and ensure that our framework is robust, additional tests should be performed such as end-to-end testing, integration testing, smoke testing and unit testing as described in [10]. These tests can be performed in DEV and TEST environments before going live in PROD. To the best of our knowledge, there is not any prior work for testing the robustness, fault-tolerance or system recovery of Machine Learning Operations framework in mixed Edge and Cloud computing settings. As future work, we would adapt works done in similar settings to our framework. For example,

[40] proposed to evaluate the response time, the availability, throughput and reliability for the Cloud setting. In the Edge setting, [41] measured different success rates of meeting loss-tolerance requirements under increasing workload to evaluate their system. In an Edge-Cloud Architecture as in [42], authors reported measurements of latency values and throughput by considering various Real-case scenarios for assessing fault tolerance. An additional consideration should also be made as Machine Learning systems are inherently different from classical software engineering systems. In other terms, in Machine Learning systems, there exist additional technical debts to be paid. An exploration of several ML-specific risk factors (e.g. data dependency, data entanglement, feedback loops...) are described in [43] and can serve as robustness and fault-tolerance evaluation metrics.

B. Considerations on security and privacy

In the current version of our framework, security and privacy challenges are not addressed and can be an interesting future direction of our work. Cloud and Edge computing paradigms are not perfect as they are vulnerable to attacks and raise concerns on privacy. In our framework, deploying models on the edge gives the advantage of moving the computation closer to the device itself. As a result, it gives faster responsiveness and processing compared to cloud-based solutions. In addition, Edge computing allows to filter out sensitive data which has to be sent to the cloud and adding a security level for the data. However, Edge computing is also vulnerable to some security and privacy attacks such as Eavesdropping, Denial of Service, Data Tempering, exploiting weak credentials for protection or using insecure communication [44]. For each attack, there exist countermeasures and additional work can be done on that to propose a more secure MLOps framework. To ensure privacy on users' sensitive information a countermeasure proposed in [45] is to distribute the sensitive information through Edge computing nodes such that no node has complete knowledge of the information. As an open research direction, the authors suggest to investigate federated learning strategies in case of Machine Learning application on the Edge.

C. Framework extension for Federated Learning

Data privacy may be necessary, for instance, in Healthcare where in accordance with regulations like the GDPR [46], patient data cannot be used as it is and personally identifiable information should be anonymised. To tackle this challenge, a Federated Learning approach can be implemented by extending the proposed framework as we show in Figure 7.

Federated Learning [47] is a way of performing Machine Learning in a collaborative fashion. The training process is distributed across multiple edge devices storing only a local sample of the data. Data is not exchanged nor transferred between edge devices or the cloud to maintain data privacy and security. In our framework, data storage and model training are decentralised and fine-tuning of ML models is centralised on the cloud. The ML pipeline step of the Cloud Orchestration layer, does not ingest data anymore but rather the ML models

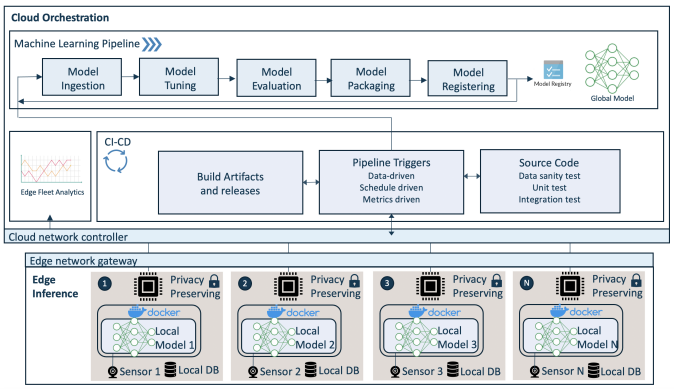

Fig. 7. Edge MLOps Framework for Federated Learning

are then trained locally. This process calibrates the global models that are then audited and stored in the model registry. These fine-tuned global models are then only stored data on the cloud. We leave the experimental validation of this extension as a future work.

IX. Conclusion

Edge MLOps is a framework combining cloud and edge environments for operationalising ML models. Cloud solutions propose high computation power and storage, while edge environments reduce latency and network connectivity dependence.

We proposed a complete lifecycle for automating AIoT workloads by orchestrating two main pipelines (cloud orchestration and edge inference) to ensure modularity. Each pipeline is in turn composed of other pipelines such as ML pipeline, CI-CD pipeline, Continuous Deployment pipeline at the edge etc. Traditionally each pipeline is executed separately or connected together in an adhoc manner. In this paper, we presented an approach to synchronise all these steps for realtime automation of cloud and edge operations. Our framework was successfully implemented in a real-life scenario where the goal was to deploy ML models to forecast the air quality of rooms directly on the edge. It integrates features such as different pipeline triggers (e.g. data drift, model versioning, model drift), is language and library agnostic (based on containers) and can be easily extended in a Federated Learning setting.

As future work, we would investigate the behaviour and the performance of our framework on cases with a more time-sensitive AI setup, such as a short training time-frame. Moreover, as data privacy and security are becoming increasingly necessary, experimentally validating our framework in the Federated Learning setting is a natural way forward.

Acknowledgments

This work is supported by TietoEVRY and the 5G-FORCE project "6388/31/2018 5G-FORCE" (www.5g-force.org).

References

[1] K. Bilal, O. Khalid, A. Erbad, and S. Khan, "Potentials, trends, and prospects in edge technologies: Fog, cloudlet, mobile edge, and micro data centers," *Computer Networks*, vol. 130, 10 2017.

[2] Y. Wu, Y. Wu, and S. Wu, *An outlook of a future smart city in Taiwan from post–Internet of things to artificial intelligence Internet of things*. Elsevier, 01 2019, pp. 263–282.

[3] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, pp. 1–1, 10 2016.

[4] G. Agarwal, *"What is Edge AI and how it fills the cracks of IoT*, 2019.

[5] M. Westerlund, "A study of EU data protection regulation and appropriate security for digital services and platforms," Ph.D. dissertation, Åbo Akademi University, 2018.

[6] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtarik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," in *NIPS Workshop on Private Multi-Party Machine Learning*, 2016.

[7] M. Beck, M. Werner, S. Feld, and T. Schimper, "Mobile edge computing: A taxonomy," in *The Sixth International Conference on Advances in Future Internet (AFIN 2014)*, 01 2014.

[8] J. Wickström, M. Westerlund, and G. Pulkkis, "Smart contract based distributed IoT security: A protocol for autonomous device management," *In proceedings of 21st ACM/IEEE International Symposium on Cluster, Cloud and Grid Computing (CCGrid 2021) (forthcoming)*, 2021.

[9] K. Sato, "What is ML Ops? Best practices for DevOps ML," 2018, Presentation at Google Cloud Next'18. [Online]. Available: https://cloud.withgoogle.com/next18/sf/sessions/session/192579

[10] E. Raj, *Engineering MLOps*. Birmingham, United Kingdom: Packt, 2021.

[11] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "Distributed deep neural networks over the cloud, the edge and end devices," in *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2017, pp. 328–339.

[12] D. A. Tamburri, "Sustainable mlops: Trends and challenges," in *2020 22nd International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)*, 2020, pp. 17–23.

[13] S. B. Calo, M. Touna, D. C. Verma, and A. Cullen, "Edge computing architecture for applying ai to iot," in *2017 IEEE International Conference on Big Data (Big Data)*, 2017, pp. 3012–3016.

[14] J. Chen and X. Ran, "Deep learning with edge computing: A review," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1655–1674, 2019.

[15] C. Wu, D. Brooks, K. Chen, D. Chen, S. Choudhury, M. Dukhan, K. Hazelwood, E. Isaac, Y. Jia, B. Jia, T. Leyvand, H. Lu, Y. Lu, L. Qiao, B. Reagen, J. Spisak, F. Sun, A. Tulloch, P. Vajda, X. Wang, Y. Wang, B. Wasti, Y. Wu, R. Xian, S. Yoo, and P. Zhang, "Machine learning at facebook: Understanding inference at the edge," in *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2019, pp. 331–344.

[16] S. Wang, C. Ding, N. Zhang, X. Liu, A. Zhou, J. Cao, and X. S. Shen, "A cloud-guided feature extraction approach for image retrieval in mobile edge computing," *IEEE Transactions on Mobile Computing*, 2019.

[17] Y. Lee, A. Scolari, B.-G. Chun, M. Weimer, and M. Interlandi, "From the edge to the cloud: Model serving in ml. net." *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 46–53, 2018.

[18] B. Sudharsan, J. G. Breslin, and M. I. Ali, "Edge2train: a framework to train machine learning models (svms) on resource-constrained iot edge devices," in *Proceedings of the 10th International Conference on the Internet of Things*, 2020, pp. 1–8.

[19] Gartner, *Integrate DevOps and Artificial Intelligence to Accelerate IT Solution Delivery and Business Value*, 2017, https://www.gartner.com/doc/3787770/integrate-devops-artificial-intelligence-accelerate.

[20] J. Hermann and M. D. Balso, *Meet michelangelo: Ubers machine learning platform*, 2017, https://eng.uber.com/michelangelo.

[21] M. Ali and Y. Lee, "Crm sales prediction using continuous time-evolving classification," in *AAAI*, 2018.

[22] J. Ereth, "Dataops-towards a definition." *LWDA*, vol. 2191, pp. 104–112, 2018.

[23] A. R. Munappy, D. I. Mattos, J. Bosch, H. H. Olsson, and A. Dakkak, "From ad-hoc data analytics to dataops," in *Proceedings of the International Conference on Software and System Processes*, 2020, pp. 165–174.

[24] P. Agrawal and N. Rawat, "Devops, a new approach to cloud development testing," in *2019 International Conference on Issues and Challenges in Intelligent Computing Techniques (ICICT)*, vol. 1, 2019, pp. 1–4.

[25] H. Tian, M. Yu, and W. Wang, "Continuum: A platform for cost-aware, low-latency continual learning," in *Proceedings of the ACM Symposium on Cloud Computing*, ser. SoCC '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 26–40.

[26] J. Moon, S. Kum, and S. Lee, "A heterogeneous iot data analysis framework with collaboration of edge-cloud computing: Focusing on indoor pm10 and pm2. 5 status prediction," *Sensors*, vol. 19, no. 14, p. 3038, 2019.

[27] A. Bhattacharjee, Y. Barve, S. Khare, S. Bao, Z. Kang, A. Gokhale, and T. Damiano, "Stratum: A bigdata-as-a-service for lifecycle management of iot analytics applications," in *2019 IEEE International Conference on Big Data (Big Data)*, 2019, pp. 1607–1612.

[28] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, ser. Springer Series in Statistics. New York, NY, USA: Springer New York Inc., 2001.

[29] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *J. Mach. Learn. Res.*, vol. 13, no. null, p. 281–305, Feb. 2012.

[30] E. Raj, M. Westerlund, and L. Espinosa-Leal, "Reliable fleet analytics for edge iot solutions," *Cloud Computing 2020: The Eleventh International Conference on Cloud Computing, GRIDs, and Virtualization*, p. 55, 2020.

[31] R. Akkiraju, V. Sinha, A. Xu, J. Mahmud, P. Gundecha, Z. Liu, X. Liu, and J. Schumacher, "Characterizing machine learning process: A maturity framework," *arXiv preprint arXiv:1811.04871*, 2018.

[32] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of things: A survey on enabling technologies, protocols, and applications," *IEEE Communications Surveys Tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015.

[33] R. Wieringa, *Design science methodology for information systems and software engineering*. Springer, 2014, 10.1007/978-3-662-43839-8.

[34] R. J. Hyndman and G. Athanasopoulos, *Forecasting : principles and practice*, 2nd ed. OTexts.com, 2014.

[35] D. F. Andrews, "A robust method for multiple linear regression," *Technometrics*, vol. 16, no. 4, pp. 523–531, 1974.

[36] A. Akusok, K.-M. Björk, Y. Miche, and A. Lendasse, "High-performance extreme learning machines: a complete toolbox for big data applications," *IEEE Access*, vol. 3, pp. 1011–1025, 2015.

[37] A. Liaw, M. Wiener *et al.*, "Classification and regression by randomforest," *R news*, vol. 2, no. 3, pp. 18–22, 2002.

[38] Microsoft, "Yearly running cost per rasbpi - raspberry pi forums," https://www.raspberrypi.org/forums/viewtopic.php?t=18043, (Accessed on 04/12/2021).

[39] "Pricing - windows virtual machines — microsoft azure," https://azure.microsoft.com/en-us/pricing/details/virtual-machines/windows/, (Accessed on 04/12/2021).

[40] P. Kumari and P. Kaur, "A survey of fault tolerance in cloud computing," *Journal of King Saud University - Computer and Information Sciences*, 2018.

[41] C. Wang, C. Gill, and C. Lu, "Frame: Fault tolerant and real-time messaging for edge computing," in *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*, 2019, pp. 976–985.

[42] A. Javed, J. Robert, K. Heljanko, and K. Främling, "Iotef: A federated edge-cloud architecture for fault-tolerant iot applications," *Journal of Grid Computing 18*, vol. 18, 2020.

[43] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2*, 2015, p. 2503–2511.

[44] S. Parikh, D. Dave, R. Patel, and N. Doshi, "Security and privacy issues in cloud, fog and edge computing," *Procedia Computer Science*, vol. 160, pp. 734–739, 2019.

[45] A. Alwarafy, K. A. Al-Thelaya, M. Abdallah, J. Schneider, and M. Hamdi, "A survey on security and privacy issues in edge-computing-assisted internet of things," *IEEE Internet of Things Journal*, vol. 8, no. 6, pp. 4004–4022, 2021.

[46] P. Voigt and A. Von dem Bussche, "The eu general data protection regulation (gdpr)," *A Practical Guide, 1st Ed., Cham: Springer International Publishing*, 2017.

[47] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, Jan. 2019. [Online]. Available: https://doi.org/10.1145/3298981

View publication stats